OPERATING SYSTEMS PROJECT
REVIEW 2 — METHODOLOGY AND PARTIAL IMPLEMENTATION (LAB ASSESSMENT 9)

Adaptive CPU Scheduling Using Reinforcement Learning for Dynamic Workloads

Self-healing scheduling in a discrete-event CPU simulator

Causal arrived-work observation • monitor, detect, diagnose, act, verify • rule-based and tabular Q-learning recovery • accepted, rolled back or safe-fallback settlement • seven seeded workload families • held-out crossed-bootstrap evaluation with a counterfactual audit

Field	Value
Project title	Adaptive CPU Scheduling Using Reinforcement Learning for Dynamic Workloads
Project system	Self-healing CPU scheduler in a discrete-event simulator (monitor, detect, diagnose, act, verify, settle, record)
Review stage	Review 2 — Methodology and Partial Implementation (Lab Assessment 9)
Repository	github.com/lostpilot404/OS-Project
Language / libraries	Python 3.11.2 · NumPy 2.4.6 · pandas 3.0.6 · Matplotlib 3.11.2 · pytest 9.1.1
Automated verification	test suite not re-run for this build
Primary evidence	210 held-out traces per arm (final-test seed 23819); 973 audited interventions
Report generated	2026-10-16

10	8	648 × 8	65	472
lifecycle stages implemented	modules with workflows	causal states × recovery actions	held-out interventions verified as improvements	automated tests passing

Summary of the work presented. The project replaces a fixed CPU scheduling policy with a **self-healing scheduler**: a discrete-event simulator runs a seeded workload while a controller monitors realized waiting times from completed work only, declares a degradation event against the trace's own stable reference, diagnoses a likely cause from the visible ready set, asks a recovery policy — a predeclared rule or a tabular Q-learning agent — for one of 8 actions, applies it, verifies the result over a declared observation window, and then accepts the change, rolls it back, or pins a safe fallback configuration. Every decision is recorded with the evidence behind it and is audited after the run by a matched counterfactual replay. On the frozen held-out split (210 traces per arm, seed 23819) the waiting-time rule and the learned controller both beat the no-action control by 8.45 waiting-time units, the learned controller is statistically indistinguishable from the rule (0.18 [-0.04 , 0.48]), and neither reaches fixed SJF (94.66 versus 122.56 waiting-time units). The report documents the methodology (§1), architecture (§2), modules and their workflows (§3–4), dataset and telemetry (§5), tools and algorithms (§6), the experimental plan with its results (§7), the implementation progress with real outputs (§8), and the challenges that remain (§9), including the measured limit of single-window verification.

Every number in this report is read at build time from the committed artifacts under results/healing (and, for the preserved secondary workflows, results/study and results/runtime); the figures are rendered from the same files. Reproduce this document with the two commands in Appendix A.1.

Contents

The report follows the Review 2 rubric item by item: the nine numbered headings below are the nine required items, followed by an appendix of reproduction commands, seeds and preserved workflows.

1. Detailed methodology of the proposed system 4

1.1 Research questions . . . 4

1.2 The control loop . . . 4

1.3 The causal observation contract . . . 5

1.4 State abstraction, action space and learning objective . . . 5

1.5 Verification as a declared test, not an opinion . . . 6

1.6 Methodological controls . . . 6

1.7 Scope and assumptions . . . 7

2. Overall conceptual, architectural and pipeline diagram 7

3. Description of the major modules 8

4. Workflow diagram for each major module 9

5. Dataset details and data collection procedure 12

5.1 Families . . . 12

5.2 Splits and seeds . . . 13

5.3 What is collected during a run, and when . . . 13

5.4 Measured dataset statistics . . . 13

5.5 Integrity of the dataset . . . 14

6. Tools, technologies, algorithms and frameworks used 14

6.1 Tools and technologies . . . 14

6.2 Algorithms . . . 14

6.3 The control loop in pseudocode . . . 15

6.4 Literature the design draws on . . . 15

7. Experimental plan and evaluation metrics 16

7.1 Experimental plan . . . 16

7.2 Evaluation metrics . . . 16

7.3 Statistical procedure . . . 17

7.4 Held-out results . . . 17

7.5 What the settlement stage actually did . . . 18

7.6 Verification quality on the held-out interventions . . . 19

7.7 Detector quality . . . 20

7.8 Learning coverage . . . 20

8. Implementation progress 20

8.1 Completion by lifecycle stage . . . 20

8.2 Automated verification of the implementation . . . 21

8.3 Demonstration 1 — the settlement stage on real runs . . . 21

8.4 Demonstration 2 — one trace, decision by decision . . . 21

8.5 Demonstration 3 — artifacts produced by one command . . . 22

9. Challenges encountered and plan for completing the remaining work 23

9.1 Challenges encountered . . . 23

9.2 What would move the needle, in order of expected value . . . 23

9.3 Sequence for the remaining work . . . 24

Appendix A — Commands, seeds and preserved workflows 24

A.1 Exact reproduction commands 24

A.2 Seeds in force 24

A.3 Preserved secondary workflows 25

1. Detailed methodology of the proposed system

The proposed system is a **self-healing CPU scheduler**. A discrete-event simulator executes one workload trace on a single CPU with a persistent FIFO ready queue; a controller watches the performance that the trace actually achieves, decides when that performance has degraded, works out why from evidence it can observe, changes one scheduler configuration, and then has to justify the change with evidence gathered *after* it was made. The loop is closed by a settlement stage that keeps the change, undoes it, or abandons adaptation altogether for the rest of the trace. Nothing in the loop may look at unarrived work, at the finished schedule, or at any counterfactual outcome.

1.1 Research questions

- **1. RQ1 (causality).** Can a scheduler detect that *it* is performing badly using only telemetry a real system would have at that moment — arrival and completion notices of arrived work — without being handed the workload?
- **2. RQ2 (usefulness).** Do the recovery actions improve realized waiting time relative to the same monitor that never acts (the detection-only control)?
- **3. RQ3 (competitive strength).** Does the self-healing loop reach the strongest fixed policy on these workloads, SJF, which the simulator allows as an oracle because burst times are visible?
- **4. RQ4 (learning).** Does a tabular Q-learning controller add value over a predeclared hand-written recovery rule that sees exactly the same evidence?
- **5. RQ5 (verification).** Can the system establish, within the trace, that an intervention helped — or must it report that the evidence was insufficient?
- **6. RQ6 (detector quality).** How often does the detector fire on a stable workload, and how reliably does it fire on an injected degradation?

1.2 The control loop

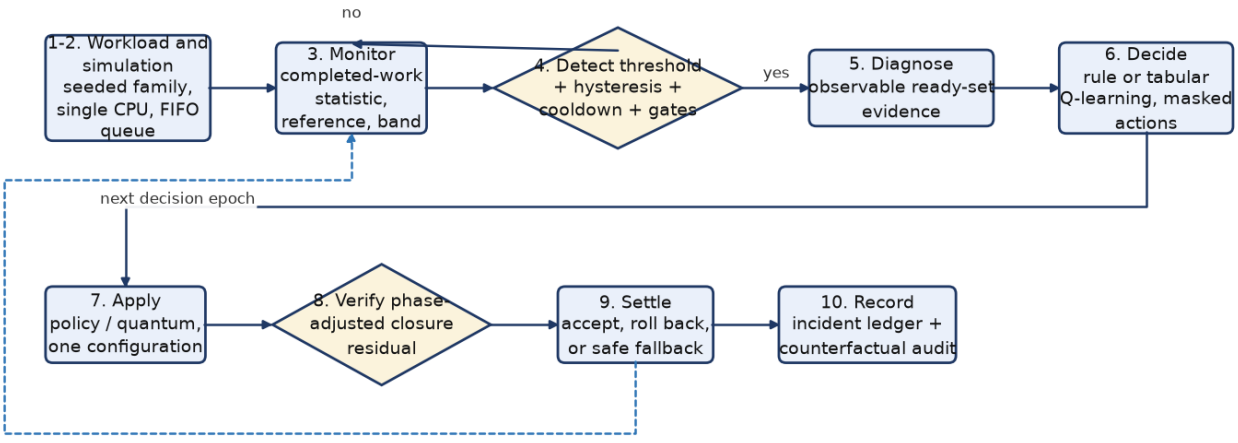


Figure 1 — The ten declared stages. Stages 3–9 run at every incident; stage 10 runs once per intervention and once per experiment.

- **1. Generate (1).** A workload is drawn from one of seven seeded families (§5). Seeds are derived per split, so no trace is ever shared between splits.
- **2. Simulate (2).** The engine admits arrivals at their times, dispatches the head of a single FIFO ready queue under the configuration in force, charges one unit of switching cost when the running process changes, and collects completions.
- **3. Monitor (3).** Every completed process contributes its realized waiting time to a rolling window of the last 8 completions, compared against an exponentially smoothed reference and dispersion band; a separate statistic tracks the waiting-time accrual rate of the ready queue.
- **4. Detect (4).** A degradation event requires the window mean to exceed *reference* + *1.25* × *band* for 2 consecutive evaluations, at least 3 decisions after the previous incident, enough visible work (≥ 2 ready processes and at least 1.5 ×

the observed mean burst of remaining CPU work), a queue at least as congested as the trace's own normal level, and an unused intervention budget.

- **5. Diagnose (5).** The visible ready set is mapped, by a fixed precedence order, to one primary cause with the evidence that matched it.
- **6. Decide (6).** A recovery policy chooses one of 8 actions from the masked set of actions that can actually change the configuration.
- **7. Apply (7).** The configuration — dispatch policy and Round-Robin quantum — is applied to the next dispatch. An action that cannot change anything is reported as a hold and never counted as an intervention.
- **8. Verify (8).** A verification window closes after 5 decisions and 3 completions, or at 14 decisions, and yields *accepted*, *rejected* or *inconclusive*.
- **9. Settle (9).** An accepted change becomes the new known-good configuration; a rejected change is confirmed harm and is rolled back; an inconclusive change is rolled back as well but is not treated as a failure; two consecutive confirmed failures disable adaptation and pin the last accepted configuration for the remainder of the trace.
- **10. Record (10).** The incident ledger stores the trigger evidence, the diagnosis, the action, the window statistics, the verdict, the realized switch overhead and the reward.

1.3 The causal observation contract

The simulator owns the future; the controller does not. This is enforced by construction rather than promised: the controller interface accepts a *RuntimeObservation* — the arrived, unfinished ready set with arrival times, remaining bursts and priorities, counters of arrived and completed processes, and the observed mean burst — plus the completion notices of the interval that just ended. It is never given the workload object, future arrivals, the finished schedule, or the outcome of any counterfactual. The test suite pins this boundary: one test changes unarrived processes and asserts that the observation and the action prefix stay identical, and another asserts that an arrival after a window closed cannot change the verdict that window already produced.

Burst times are visible once a process has arrived. That is an assumption of the study, not a property of real kernels: it is what makes SJF and the diagnosis features implementable here, and it is reported as a limitation in §9.

1.4 State abstraction, action space and learning objective

The learned controller sees a coarse, fully causal encoding of the incident. The five features and their bins are declared in *healing/policies.py*:

Feature	Bins	Bin count
Primary diagnosis	head-of-line blocking, fairness starvation, priority anomaly, tail dispersion, queue congestion, unexplained	6
Excess over threshold	< 1.5k, < 2.5k, >= 2.5k band widths (k = detection multiplier)	3
Ready-queue size	1-2, 3-4, 5 or more	3
Policy in force	FCFS, SJF, Round Robin, Priority	4
Visible remaining work	< 3, < 8, >= 8 multiples of the reference waiting time	3

Table 1 — The five causal state features.

The state space is the mixed-radix product 6 × 3 × 3 × 4 × 3 = **648 states**, and with 8 actions the table holds 5184 entries, small enough to be dumped in full as an artifact.

- **Actions.** `hold`, `set_fcfs`, `set_sjf`, `set_round_robin`, `set_priority`, `shrink_quantum`, `grow_quantum`, `revert_to_last_accepted`.
- **Masking.** Actions that cannot change the live configuration (a policy already in force, a quantum change under a non-Round-Robin policy, a revert without an earlier accepted configuration) are removed before the policy is asked, so a no-op can never be recorded or rewarded as a recovery.
- **Reward.** *reward = phase-adjusted residual − accept boundary − intervention cost*, minus a declared rollback penalty only when the window is a **confirmed** failure. The residual is the same statistic the verdict is taken on, so the learner is never rewarded for something the verifier could not see; the intervention cost is 1% of the reference waiting time per change and the rollback penalty 2%. A hold is scored on exactly the same scale, so doing nothing wins whenever the statistic does not exceed the boundary by more than the cost of acting.

- **Q-learning.** Learning rate $\alpha = 0.10$, discount $\gamma = 1.00$ (the recovery decisions of one trace form one episodic return), ε from 1.00 decaying by 0.997 per episode to a floor of 0.05, zero-initialised values, greedy and read-only during evaluation, with a counted deterministic fallback in a state that has no visited action.

1.5 Verification as a declared test, not an opinion

Verification answers one question at the end of a window: *does the evidence recorded so far justify keeping this configuration?* The statistic is the normalised load closure of the window — the fraction of the gap between the queue level observed at the trigger and the trace's own normal level that the window closed:

```
closure  = (trigger_load - post_load) / max(trigger_load - reference_load, load_band)
expected = baseline(trigger_load, ready_count, log1p(visible_work))  -> closure units
residual = closure - expected
```

Listing 1 — The verification statistic.

The comparison is made against a **phase baseline** rather than against a single constant, because a trigger is by construction a local maximum of the monitored statistic and the queue then drains whether or not anything changed. The baseline predicts the post-window queue level that *no intervention* would have produced, from three quantities observed at the trigger (the trigger's own queue level, the ready-set size, and the logarithm of the visible remaining work), and that prediction is turned into closure units with the statistic's own denominator. On the training control this removes 80% of the statistic's no-action variance (baseline $R^2 = 0.961$, residual spread 0.160 in closure units, over 6000 training traces).

The two boundaries are percentile points of the residual distribution over the windows the rule can actually judge in the no-action control of the **training** split (3633 windows): the 0.263 95th percentile becomes the declared accept boundary **0.30** and the -0.257 5th percentile becomes the reject boundary **-0.30**, both rounded outward. The measured false-accept and false-reject rates are 3.4% and 3.1% on that control, and 2.1% and 1.1% out of sample on the control of the validation split.

Verdict	Condition	Effect on the configuration	Effect on learning and safety
accepted	residual ≥ accept boundary	kept; becomes the last accepted configuration	scored on the same statistic; the failure counter resets
rejected	residual ≤ reject boundary	rolled back to the last accepted configuration	carries the declared rollback penalty and increments the failure counter
inconclusive	between the boundaries, thin telemetry, or a truncated window	rolled back because *inconclusive_rollback* = *True*	no penalty and no failure count: ambiguity can neither teach the controller that an action was harmful nor trigger the fallback
held	no configuration change was made	nothing to undo	scored on the same statistic, so holding competes with acting

Table 2 — The four settlement outcomes.

1.6 Methodological controls

- **Identical workloads.** Every arm of the study runs the same workload instances; differences are paired per trace, and a fingerprint check asserts that the splits share no instance.
- **Blinded held-out split.** Training seeds 7101, 7102, 7103, 7104, 7105, validation seed 8201, and a final-test seed **23819** declared before the frozen run. The configuration refuses any retired stream as a held-out seed, and the threshold derivation refuses the final-test split outright; all thresholds and boundaries in this report come from the training control and are checked out of sample on validation.
- **Retired streams.** Six earlier seed streams were each examined while the system was still changing; they are recorded as retired and are never quoted as held-out evidence.
- **Uncertainty.** Differences are family-stratified crossed bootstrap intervals (1000 replicates, seed 104729) over model seeds and workload traces.
- **Counterfactual audit instead of an in-run oracle.** For every intervention, the same controller is re-run on the same workload with the same recorded prefix and exactly that one incident forced to hold; the audit verifies that the two runs are identical up to the incident and diverge there. The audit is computed after the run and is never visible to the controller.
- **Determinism.** Two independent runs of the frozen configuration reproduce all 21 machine-readable artifacts and all three figures byte for byte. Wall-clock timings are recorded but never used as a result and never charged to simulated

time.

1.7 Scope and assumptions

The project studies one question — can a CPU scheduler detect that its own configuration is failing and recover from causal evidence alone? — under a set of assumptions that are stated here so the results can be read correctly.

- **In scope.** A single CPU; one FIFO ready queue; non-preemptive service under FCFS, SJF and Priority; quantum-bounded service under Round Robin; one unit of switching cost per change of running process; synthetic workloads drawn from the seven declared families; an online controller that may act only on arrived work; and an offline audit that measures what each intervention actually achieved.
- **Out of scope.** Real operating-system kernels and hardware. Nothing in this repository is a kernel module, a syscall, a patch to a real scheduler or a measurement taken on a real machine, and no result here should be read as one. Also out of scope: multiple CPUs, hyper-threading, I/O and page-fault behaviour, interrupt accounting, priority inheritance or a real clock tick.
- **Assumptions.** Burst times become visible when a process arrives (which is what lets SJF, and the diagnosis features, exist at all); arrivals, bursts and priorities are drawn from declared distributions; the trace is finite; and the performance statistic the controller watches is the realized waiting time of completed work, which a real scheduler can measure from its own dispatch and completion events.
- **What a good result would look like.** The loop must beat the no-action control on identical traces, must not damage a configuration that is already good, and must not claim a benefit it cannot evidence. The current release satisfies the first two and states honestly where it fails the third for individual interventions (§7.5–7.6).

2. Overall conceptual, architectural and pipeline diagram

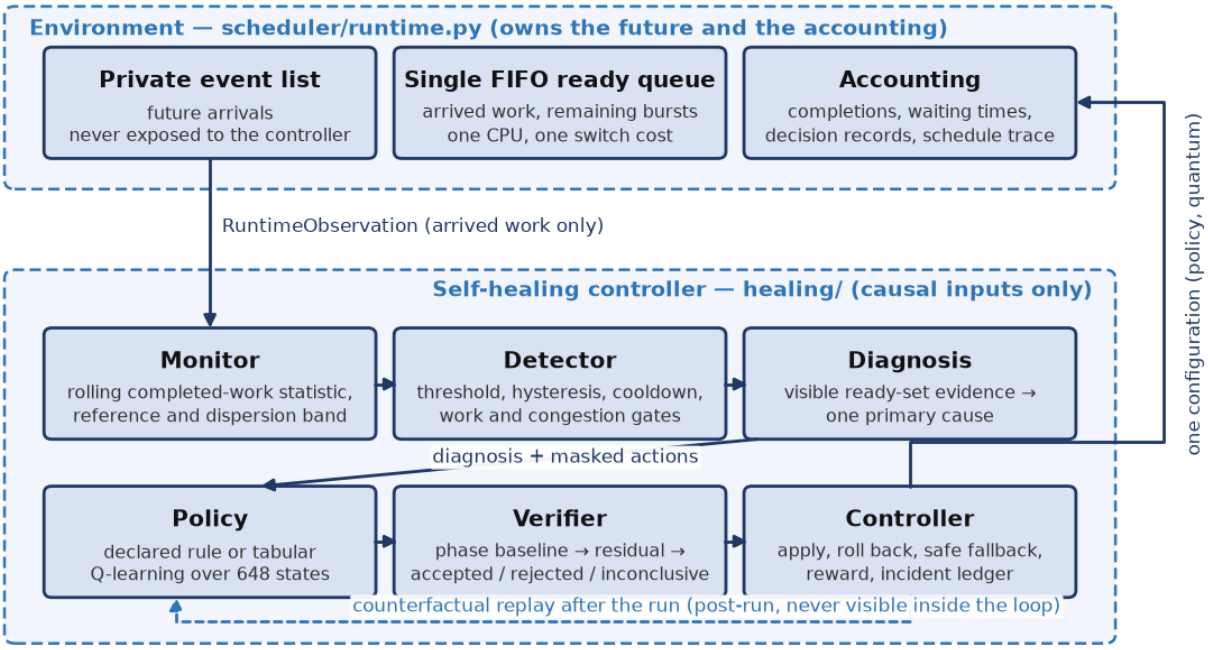


Figure 2 — System architecture. The environment owns the future and the accounting; the healing loop receives observations of arrived work and replies with one configuration.

The architecture separates three responsibilities that the earlier version of this project did not separate: a **machine** that owns time, arrivals and accounting; a **control loop** that may act only on causal evidence; and an **evidence layer** that records what happened and audits it afterwards.

- **Environment (*scheduler/runtime.py*).** Owns the private event list, the single FIFO ready queue, remaining bursts, switching cost and the schedule trace. It freezes an immutable observation at every decision epoch and accepts exactly one reply: the configuration to use next.
- **Perception (*healing/monitor.py*, *healing/detector.py*).** Turns completion notices and ready-queue accrual into a rolling statistic with a stable reference and dispersion band, and decides whether the trace is degraded.

- **Reasoning (*healing/diagnosis.py*, *healing/actions.py*, *healing/policies.py*).** Names a likely cause from visible evidence, defines the eight recovery actions and their no-op semantics, and selects one action with a rule or a learned controller.
- **Judgement (*healing/verifier.py*, *healing/controller.py*).** Turns a closed observation window into a verdict, and turns a verdict into a configuration decision, a failure count and a learning reward.
- **Evidence (*healing/controller.py* ledger, *experiments/healing_experiment.py*).** Records every incident with the numbers behind it and audits every intervention by counterfactual replay.

The information boundary is the architectural claim of the project. It is not that the controller *chooses not to* look at the future: the controller has no access path to it. The engine's event list is private, the controller receives a value object rather than a reference to the workload, and the replay helpers that make the counterfactual audit possible run in a separate controller instance after the trace has finished.

Layer	Owns	May observe	May change
Environment	time, arrivals, ready queue, accounting	the complete workload	nothing: it applies the configuration it is given
Healing loop	detection, diagnosis, action selection, verification	arrived ready set, completion notices, its own decisions	one configuration (policy, quantum) and, through it, future dispatches
Study harness	splits, training, statistics, audit	every artifact of a finished run	nothing during a run; it recomputes and audits afterwards

Table 3 — Who may see and change what.

3. Description of the major modules

Eight modules carry the system. M1–M2 build and run the simulated machine; M3–M7 are the healing loop; M8 is the experiment that measures it. Each module has a single declared contract, listed below with the artifact that demonstrates it.

Module	Responsibility	Files
M1	Workload model, generator and fingerprinting	workload/models.py, workload/generator.py, workload/fingerprint.py
M2	Discrete-event engine with a causal observation boundary	scheduler/runtime.py, scheduler/{fcfs,sjf,round_robin,priority}.py
M3	Causal monitor and degradation detector	healing/monitor.py, healing/detector.py
M4	Evidence-backed diagnosis and the recovery action space	healing/diagnosis.py, healing/actions.py
M5	Recovery policies: predeclared rules, hold control, tabular Q-learning	healing/policies.py, rl/q_learning.py
M6	Recovery verification	healing/verifier.py, healing/config.py
M7	Self-healing controller, ledger, rollback and safe fallback	healing/controller.py
M8	Study harness: design, audit, probe, coverage and report	experiments/healing_*.py, experiments/healing_config.py, evaluation/paired.py

Table 4 — The eight major modules.

- **M1 — Workload model, generator and fingerprinting.** Contract: *(family configuration, seed) → validated, fingerprinted workload*. Seven families differ in arrival pattern, burst distribution and priority skew; every process is validated (non-negative identifiers, arrivals and priorities, positive burst, unique PIDs) and every workload carries a content fingerprint that proves two arms were evaluated on the same instance.
- **M2 — Discrete-event engine.** Contract: *(workload, controller, configuration) → schedule trace, decision records and metrics*. One evolving FIFO ready queue; FCFS and Round Robin dispatch the head, SJF the shortest remaining burst, Priority the highest priority (the lower number is the higher priority, as declared in the scheduler configuration); non-Round-Robin service is non-preemptive and Round-Robin service is bounded by the quantum; one unit of switching cost is charged when the running process changes. The engine exposes only observations to the controller.
- **M3 — Causal monitor and degradation detector.** Contract: *(completion notices, ready-queue evidence) → degradation event with reasons, or an explicit suppression reason*. Telemetry is completion-based; a window is judged against the reference it did not create and only afterwards absorbed into it, so a permanently degraded trace keeps failing the same comparison instead of raising its own baseline. Every suppression path has its own reason string.
- **M4 — Diagnosis and the recovery action space.** Contract: *(visible ready set) → one primary cause with matched evidence*, and *(action, live configuration) → configuration change or an explicit no-op*. Six diagnosis categories are

defined by observable thresholds (head-of-line blocking, fairness starvation, priority-ordering anomaly, tail dispersion, queue congestion, unexplained).

- **M5 — Recovery policies.** Contract: *(context, allowed actions) → action with exploration bookkeeping*. A predeclared rule maps diagnosis to action; the hold-only policy is the detection control; the learned policy is a tabular Q-learning agent over the 648-state encoding, greedy and read-only during evaluation.
- **M6 — Recovery verification.** Contract: *(causal window evidence, declared thresholds) → accepted / rejected / inconclusive with the numbers behind the verdict*. The verifier also refuses to judge a window that the end of the trace truncated or that lacks the declared minimum of completions.
- **M7 — Self-healing controller and ledger.** Contract: *(observations) → configuration trajectory, incident records, reward stream*. Contains the state machine (idle → window open → verdict), the rollback at the first epoch after a verdict, the safe-fallback escalation, the reward assignment and the counterfactual replay helpers used by the audit.
- **M8 — Study harness.** Contract: *(declared design) → trained policies, held-out comparison, counterfactual audit, action probe, detector scenarios, coverage analysis, figures and summary*. The design itself — splits, seeds, arms, thresholds — is a validated configuration object, so the study can be re-run from one command.

4. Workflow diagram for each major module

Each module below is shown as a decision-level workflow. The diagrams are generated from the same declarations as the code paths they describe, and the DOCX source of this report carries the Mermaid text of each diagram.

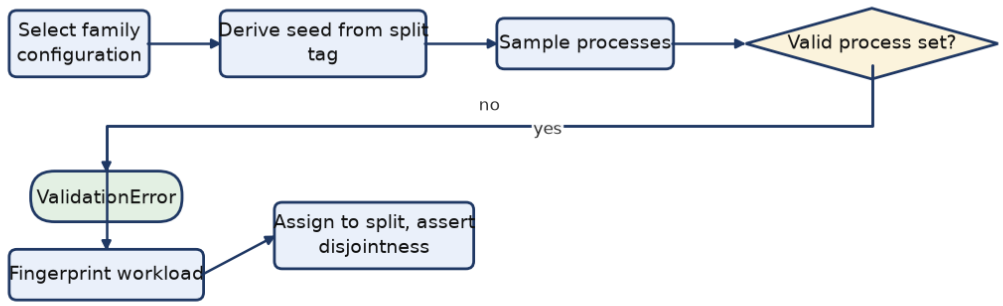


Figure 3 — M1 workload generation and split assembly: a family plus a derived seed is validated and fingerprinted before it enters a split.

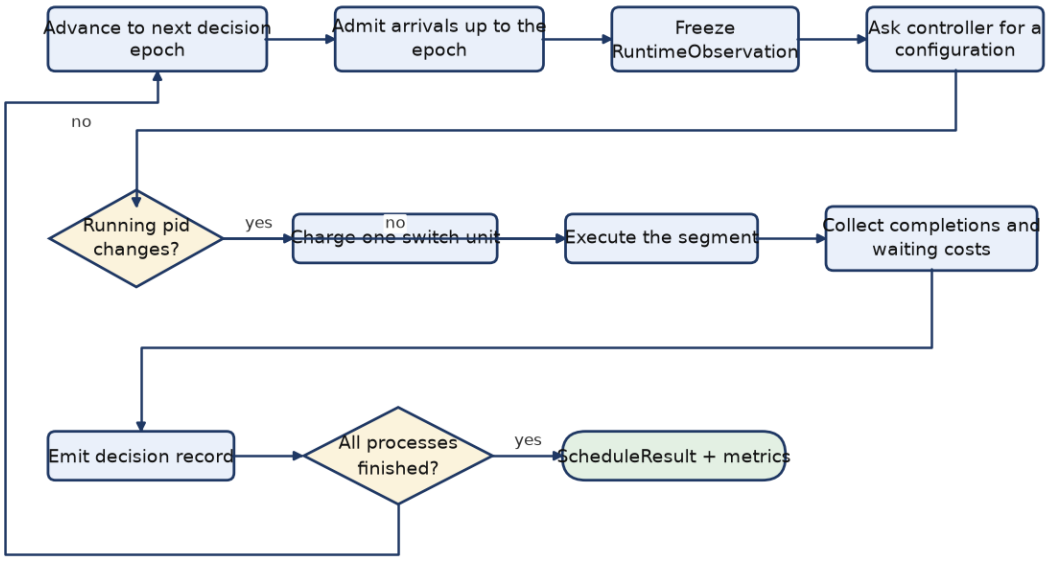


Figure 4 — M2 event engine: one decision epoch, from admitting arrivals to emitting the decision record and the interval waiting cost.

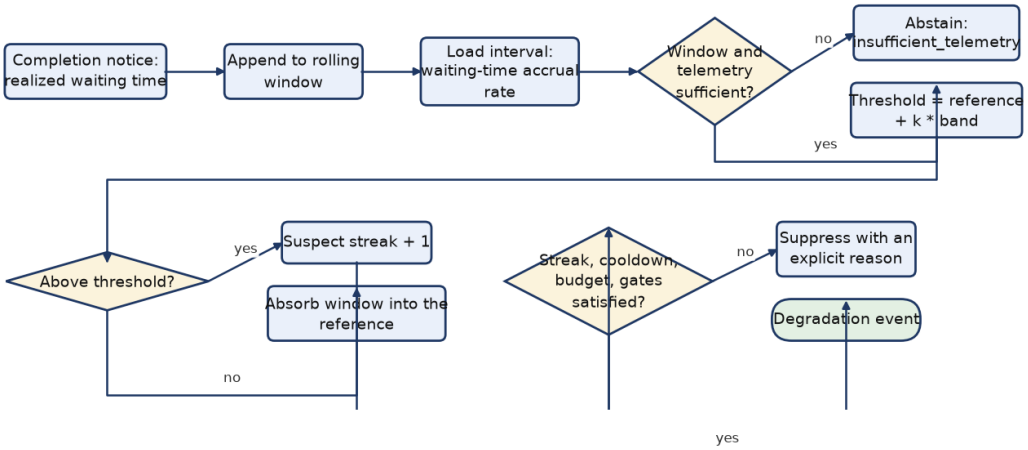


Figure 5 — M3 monitor and detector: a degradation event requires sufficient telemetry, a suspect streak, an elapsed cooldown and the work and congestion gates.

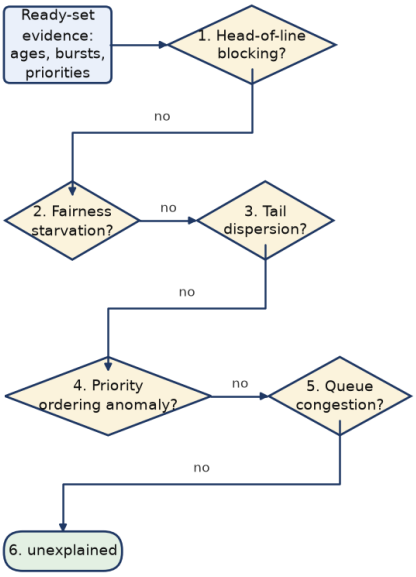


Figure 6 — M4 diagnosis: the tests are applied in this fixed precedence order and the first one that matches names the primary cause — head-of-line blocking, fairness starvation, tail dispersion, priority ordering anomaly, queue congestion, or `unexplained` when none matches.

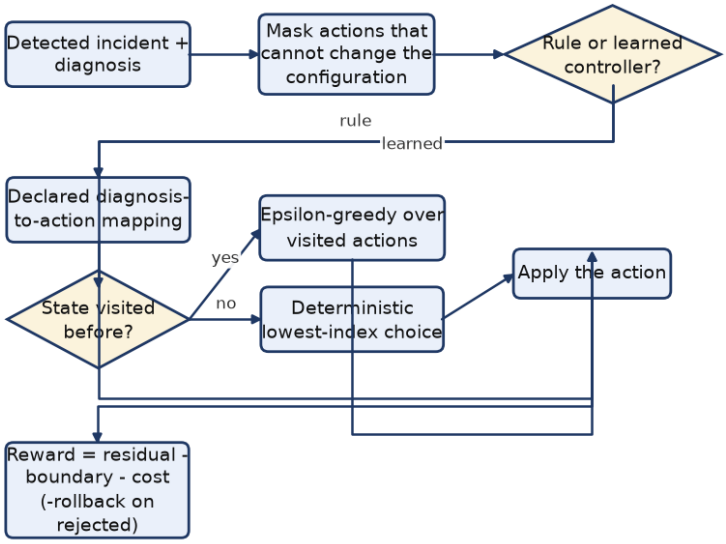


Figure 7 — M5 recovery policy: actions that cannot change the live configuration are masked before the rule or the learned controller is asked; the reward is residual minus boundary minus cost, with a rollback penalty only on a confirmed failure.

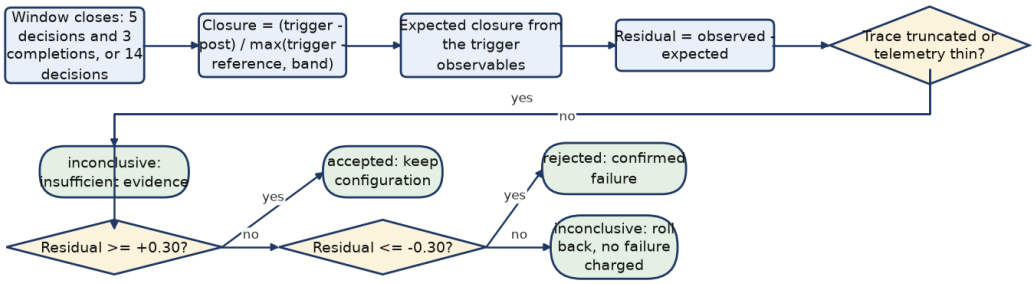


Figure 8 — M6 verifier: the window's phase-adjusted residual decides accepted, rejected or inconclusive, and never judges a truncated or thin window.

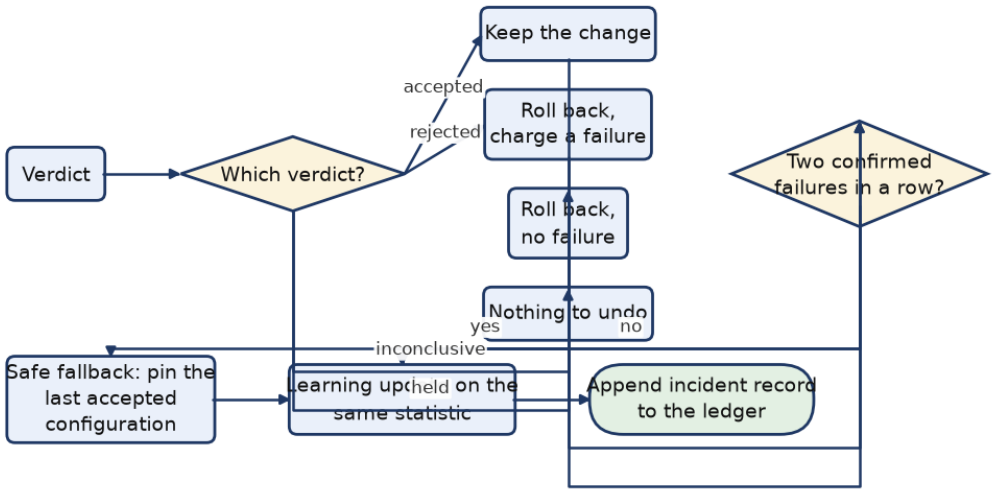


Figure 9 — M7 controller settlement: what happens to the configuration, the failure counter and the learning reward for each verdict, including the safe fallback.

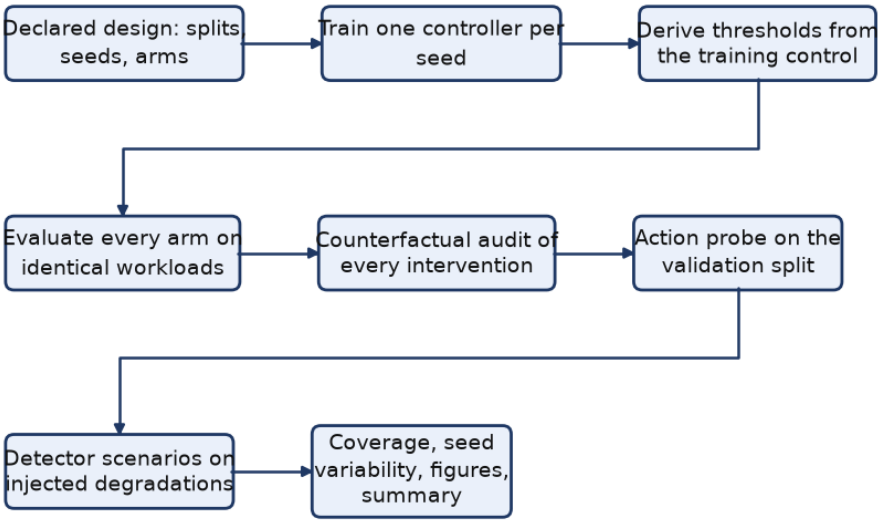


Figure 10 — M8 study harness: training, threshold derivation, arm evaluation, counterfactual audit, probe, detector scenarios and artifact export.

5. Dataset details and data collection procedure

Every workload in this project is synthetic and generated from a declared family plus a seed, which is what makes the study reproducible: a workload is an instance of a family, not a file. Each trace contains 15 processes, one CPU and one ready queue, so the interesting variation is in arrival pattern, burst distribution and priority skew rather than in trace size.

5.1 Families

Family	Arrival pattern	Burst distribution	Priority pattern	Processes
short_jobs	uniform	Short-job-heavy: 80% of bursts drawn from 1-5 time units	uniform	15
long_jobs	uniform	Long-job-heavy: 20% of bursts drawn from 1-5 time units	uniform	15
mixed	uniform	Uniform mix of bursts over the full 1-50 range	uniform	15
cpu_bursty	uniform	Long CPU bursts (20-50) with dense uniform arrivals	uniform	15
priority_skewed	uniform	Uniform bursts; 70% of priorities lie in the 1-2 band	high_priority_skewed	15
staggered_interactive	staggered	One long process at time zero plus 14 short processes arriving at integer gaps of 4-6 time units	uniform	15
poisson_arrivals	poisson	Held-out evaluation condition: Poisson arrivals (rate 0.5) and uniform bursts from 1-20	uniform	15

Table 5 — The seven workload families.

5.2 Splits and seeds

Split	Master seed	Traces	Purpose
Training	7101, 7102, 7103, 7104, 7105	1200 per seed (6000)	training the learned controllers; deriving the declared thresholds from the no-action control
Validation	8201	140	development decisions, the worked demonstration trace, and out-of-sample checks of the thresholds
Final test	23819	210	the headline comparison; evaluated once after the design was frozen

Table 6 — Splits. The final-test seed was declared before the frozen run.

Workload seeds are derived as *derive_seed(master_seed, split_tag, family index, repetition)*, so the three splits are disjoint by construction and a test asserts that no fingerprint appears in two splits. The *poisson_arrivals* family is excluded from training, so the learned controllers are evaluated on an arrival process they never saw.

5.3 What is collected during a run, and when

Data collection is deliberately minimal, because the point of the causal boundary is that the controller could not have had more. For each decision epoch the engine records the chosen process, the executed interval, the switch overhead and the identifiers and realized waiting times of the processes that completed during the interval. From those records the controller derives the rolling completion window and the waiting-time accrual rate of the ready queue.

For each incident the ledger additionally records the trigger evidence (reference, band, window statistics, excess in band widths), the diagnosis and the evidence that matched it, the offered and chosen actions with their notes, the configuration before and after, the verification window's length, completions and closure statistic, the phase baseline and residual it was judged on, the verdict and the confirmed-failure flag, the realized switch overhead and the reward. That is 70 columns per incident, which is what makes the counterfactual audit checkable rather than decorative.

5.4 Measured dataset statistics

Family	Traces	Processes / trace	Mean burst	Burst range	Max arrival span	Mean total work
cpu_bursty	50	15	35.1	20–50	15	526
long_jobs	50	15	22.8	1–50	20	343
mixed	50	15	24.5	1–50	20	368
poisson_arrivals	50	15	10.7	1–20	53	161
priority_skewed	50	15	26.4	1–50	20	396
short_jobs	50	15	8.3	1–50	20	124
staggered_interactive	50	15	4.3	1–50	76	64

Table 7 — Dataset statistics over the validation and final-test splits, computed by calling the project's own generator.

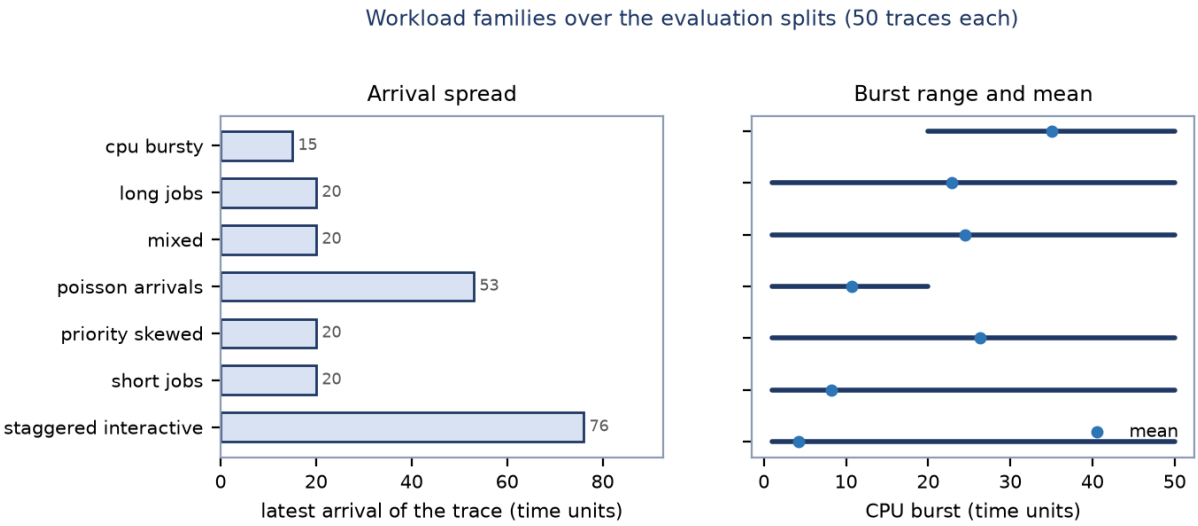


Figure 11 — Arrival span and burst range of the seven families over the evaluation splits.

5.5 Integrity of the dataset

- **Content fingerprints.** Every workload is hashed; the study asserts zero overlap between splits and uses the fingerprint to prove that paired differences are computed on identical instances.
- **Validation on construction.** A malformed process set is rejected with an exception rather than silently simulated.
- **Retired streams.** Six earlier master seeds were examined during development and are recorded as retired; the configuration refuses to reuse them as a held-out seed.
- **No hand-edited data.** Every table and figure in this report is written by *python main.py healing-study*; the report itself only reads those artifacts (plus the generator, for the dataset table).
- **Not a real workload.** No trace comes from hardware or from a production system; that limitation is carried into §9.

6. Tools, technologies, algorithms and frameworks used

6.1 Tools and technologies

Tool / library	Version	Role in the project
Python	3.11.2	implementation language of the simulator, the healing loop, the experiments and the tests
NumPy	2.4.6	seeded random sampling in the workload generator and numeric helpers
pandas	3.0.6	artifact tables, aggregation and the reported metrics
Matplotlib	3.11.2	figures of the study and of this report
pytest	9.1.1	the automated test suite (472 tests)
ReportLab / python-docx	5.0.1 / 1.2.0	report deliverables only: this PDF and the editable DOCX source, both generated from the committed artifacts

Table 8 — Tools actually used.

The platform recorded with the study is *Linux-6.1.158+-x86_64-with-glibc2.36*. No deep-learning framework, database or scientific-workflow engine is used: the whole pipeline is deterministic Python over tabular artifacts, which keeps every number in this report traceable to a file.

6.2 Algorithms

- **Exponentially smoothed reference and dispersion band** (monitor): the reference absorbs a window only after that window has been judged against the reference it did not create (reference EWMA 0.35, band EWMA 0.25, with absolute and relative floors so a regular trace cannot collapse the band and turn every deviation into an incident).
- **Reference-based detection with hysteresis, cooldown and gates**: suspect when the window mean exceeds *reference + 1.25 × band*, declared only after 2 consecutive suspects, outside the 3-decision cooldown, with enough visible CPU work and a queue at least as congested as the trace's normal level.

- **Fixed-precedence diagnosis** from observable thresholds on the ready set — a deterministic mapping rather than a learned classifier, so a diagnosis can be argued about directly.
- **Tabular Q-learning** over the 648-state encoding with masked exploration and masked bootstrapping: $\alpha = 0.10$, $\gamma = 1.00$, ε-greedy with a multiplicative decay and a floor, first-visit forcing, and greedy read-only behaviour in evaluation.
- **Least-squares phase baseline** for verification: a four-coefficient model (intercept plus the three trigger observables) fitted on the no-action control of the training split predicts the post-window queue level that no intervention would have produced ($R^2 = 0.961$ on the post-load level, and 80% of the closure statistic's no-action variance removed).
- **Percentile calibration of the decision boundaries**: the accept and reject boundaries are the 0.263 and -0.257 percentiles of the control residual, rounded outward, so the verifier's error rates are declared in advance rather than tuned after seeing the held-out split.
- **Family-stratified crossed bootstrap** for uncertainty: paired differences resampled over model seeds and workload traces (1000 replicates, seed 104729), plus a rank-based AUC without SciPy for the verifier's discrimination.
- **Matched counterfactual replay** for attribution: the same controller, same workload and same recorded prefix, with one incident forced to hold, and a divergence check that the replay is identical up to that incident.

6.3 The control loop in pseudocode

```
for each decision epoch of a trace:                      # all inputs are causal
    observation = engine.observation()                   # arrived ready set only
    record_interval(observation)                         # completions -> rolling window
    if incident_open:
        if window_ready(decisions, completions) or decisions >= max_decisions:
            verdict = verify(window_evidence, declared_thresholds)
            if verdict == accepted:  keep(configuration);        reward = residual - bound - cost
            if verdict == rejected:  rollback(); failures += 1;  reward = residual - bound - cost - penalty
            else:                    rollback();                 reward = residual - bound - cost
            if failures >= 2:        safe_fallback(); adaptation_enabled = False
        return configuration                                 # engine applies it next dispatch
    if not new_completions: return configuration             # nothing new to judge
    outcome = detector.evaluate(monitor_statistics, gates, cooldown, budget)
    if not outcome.triggered: return configuration           # reason recorded, reference updated
    diagnosis = diagnose(visible_ready_evidence)             # evidence-backed cause
    allowed   = mask(current_configuration)                  # only actions that can change something
    action    = policy.select(context, allowed)              # rule, or epsilon-greedy Q-learning
    configuration = apply(action)                            # held if the action cannot change it
    open_window(trigger_evidence, phase_baseline)            # verification starts here
```

Listing 2 — The healing loop, as implemented in healing/controller.py.

Three properties of this pseudocode matter for the report's claims. First, the only inputs are the observation, the previous interval's completions and the controller's own state. Second, the reward is computed from the same window statistic the verdict uses. Third, no branch of the loop has access to a counterfactual, to future arrivals or to the finished schedule.

6.4 Literature the design draws on

The design stands on standard, well-documented work rather than on a survey: the scheduling material establishes what a scheduler can measure and why adaptive policies exist, and the reinforcement-learning material establishes what a tabular action-value method can and cannot promise. These are the works the implemented choices follow; each is a published book or paper that a reader can verify directly.

Reference	What this project takes from it
A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, Wiley (current edition).	The vocabulary and the measurement discipline of CPU scheduling: waiting, response and turnaround time, dispatch versus completion, and the classical baselines (FCFS, SJF, Round Robin, Priority) that this project keeps as the fixed arms of its comparison.
R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books.	The scheduling chapter's framing of the scheduler's dilemma — policy choice under imperfect information — which is exactly the situation this project models by hiding the future from the controller.
P. J. Denning, “The Working Set Model for Program Behavior”, *Communications of the ACM* 11(5), 1968.	The idea that a recent, windowed view of a system is the right basis for a runtime decision, and that the window must be chosen rather than assumed — the monitor's rolling completion window and its declared reference are this project's small version of that argument.
C. A. Waldspurger and W. E. Weihl, “Lottery Scheduling: Flexible Proportional-Share Resource Management”, OSDI 1994.	The precedent that a fresh scheduling decision at runtime can be a first-class mechanism rather than a static choice, and that the fairness of such a mechanism is a property to be measured rather than assumed.
C. J. C. H. Watkins and P. Dayan, “Q-learning”, *Machine Learning* 8(3), 1992.	The update rule used verbatim in *rl/q_learning.py*, including the conditions on which its convergence result depends — conditions this project deliberately does not claim to satisfy, which is why the learned controller is reported against the rule rather than asserted to be better.
R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd edition, MIT Press, 2018.	The treatment of exploration and of episodic versus continuing returns behind the ε-greedy schedule, the first-visit fallback and the choice to make one trace one episodic return.
G. Tesauro, “Temporal Difference Learning and TD-Gammon”, *Communications of the ACM* 38(3), 1995.	The demonstration that temporal-difference learning can work well on a hard sequential decision problem with a coarse representation — and the caution that came with it, that its success depended on the problem, which is why this report measures the learned controller instead of assuming it.

Table 9 — The published work the implementation follows.

Two consequences of taking these works seriously are visible in the repository. First, the learned controller is evaluated exactly like the rule — same workloads, same seeds, paired uncertainty — instead of being assumed superior. Second, the verification stage exists at all: none of the references above offers a way to confirm, inside a single trace, that a change helped, which is the gap this project tried to close and reports as only partly closed.

7. Experimental plan and evaluation metrics

7.1 Experimental plan

Ten arm definitions are evaluated on identical workloads. Four are untouched fixed schedulers, one is the control that runs the whole monitoring, detection and diagnosis machinery but never intervenes, three are predeclared rules, and the learned controller is trained under five independent seeds. Two further arms re-run the two strongest recovery methods starting from SJF, which is the deployment question that matters: does the loop damage an already good configuration?

Arm	Type	What it is
fixed_fcfs / fixed_sjf / fixed_round_robin / fixed_priority	baseline	the standalone schedulers, no adaptation
detector_only	control	full monitoring, detection and diagnosis with a policy that never acts
rule_fairness	rule	predeclared fairness-oriented diagnosis→action rule
rule_wait	rule	predeclared waiting-time-oriented diagnosis→action rule
q_recovery	learned	tabular Q-learning recovery, one independently trained controller per seed, reported together
rule_wait_from_sjf / q_recovery_from_sjf	from-SJF	the two recovery methods started from the strongest fixed policy

Table 10 — Arms of the study.

7.2 Evaluation metrics

- **Realized scheduling metrics** computed from the finished schedule: mean, median, 95th-percentile and maximum waiting time, mean turnaround, mean response time, CPU utilization, throughput, context switches and switching overhead. A process's waiting time is *start* − *arrival* over the whole run, including every preemption.
- **Healing activity metrics**: incidents, interventions, holds, the three verdict counts, confirmed failures, interventions per trace and fallback episodes — reported per arm with explicit denominators.

- **Counterfactually measured effect**: for each intervention, the difference in total waiting time between the recorded run and a replay in which that one incident is forced to hold. A non-positive difference is what this study calls a *false recovery*.
- **Verifier quality**: the share of genuinely helpful interventions the verifier certifies, the share of accepted interventions that were helpful, and the rank AUC of the residual for separating helpful from harmful interventions.
- **Detector quality**: recall and detection delay on injected degradations, and the false-trigger rate on stable traces.
- **Learning coverage**: visited states and state-action pairs against both the declared table and the pairs reachable from the observed contexts, plus the spread of the result across independently trained seeds.

7.3 Statistical procedure

Arms that share training seeds are compared pairwise on the same traces with a family-stratified crossed bootstrap: differences are computed per workload, then resampled over seeds and traces with 1000 replicates at seed 104729, and the reported interval is the percentile interval of that resampling. Intervals are conditional on the declared design (five training seeds, seven families, fixed repetitions); they are not a claim about all workloads. Every quoted interval in this report comes from the comparisons artifact of the frozen run.

7.4 Held-out results

Arm	Mean waiting	P95 waiting	Mean response	Incidents / trace	Interventions / trace
Fixed SJF	94.66	249.20	94.66	0.00	0.00
Q-learning from SJF	94.66	249.20	94.66	0.82	0.00
Rule from SJF	94.87	249.54	94.87	0.82	0.02
Rule: waiting time	122.39	251.88	122.39	0.67	0.67
Q-learning recovery	122.56	251.98	122.54	0.67	0.65
Fixed Priority	128.32	262.75	128.32	0.00	0.00
Fixed FCFS	130.84	258.42	130.84	0.00	0.00
Detector only (control)	130.84	258.42	130.84	0.65	0.00
Rule: fairness	136.78	263.72	110.40	0.71	0.71
Fixed Round Robin	217.05	310.41	26.50	0.00	0.00

Table 11 — Held-out split, 210 traces per fixed or rule arm (learned arms: five seeds, 1050 traces). Waiting, response and turnaround times are simulated time units.

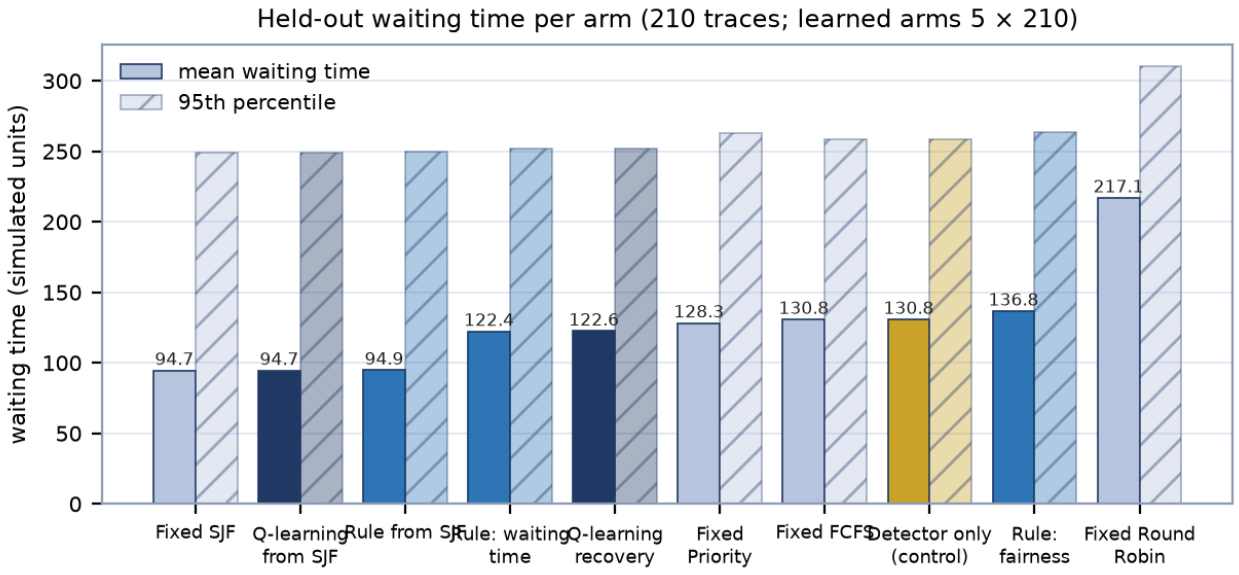


Figure 12 — Mean and 95th-percentile waiting time per arm on the held-out split.

The paired comparisons behind this table are the quantitative answer to RQ2–RQ4:

Comparison (target – reference)	Paired difference	95 % interval	Reading
rule_wait – detector_only	-8.45	[-9.44, -7.35]	the waiting-time rule beats doing nothing
q_recovery – detector_only	-8.27	[-9.33, -7.23]	the learned controller beats doing nothing
q_recovery – rule_wait	+0.18	[-0.04, +0.48]	the learned controller is indistinguishable from the rule
rule_wait – fixed_sjf	+27.73	[+26.05, +29.72]	the rule does not reach fixed SJF
q_recovery – fixed_sjf	+27.91	[+26.11, +29.87]	the learned controller does not reach fixed SJF
q_recovery_from_sjf – fixed_sjf	+0.00	[+0.00, +0.00]	started from SJF the loop changes nothing
rule_fairness – fixed_fcfs	+5.94	[+4.38, +7.45]	the fairness rule is worse than the control

Table 12 — Paired mean-waiting-time differences (1000 bootstrap replicates).

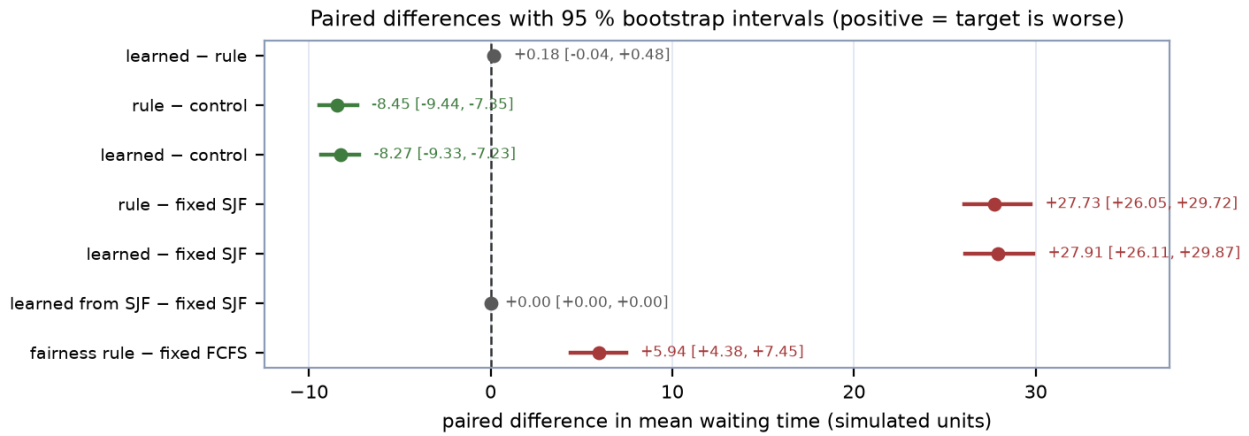


Figure 13 — Paired differences with 95 % intervals; an interval that crosses zero means the two arms are not distinguishable.

7.5 What the settlement stage actually did

Verdict counts are the in-run record: what the verifier concluded about each intervention, and what the safety machinery did as a result.

Arm	Interventions	accepted	rejected	inconclusive	fallback episodes
rule_fairness	150	2	31	117	1
rule_wait	141	12	0	129	0
rule_wait_from_sjf	4	0	1	3	0
q_recovery	678	51	2	625	0

Table 13 — Verdicts and safe-fallback episodes on the held-out split.

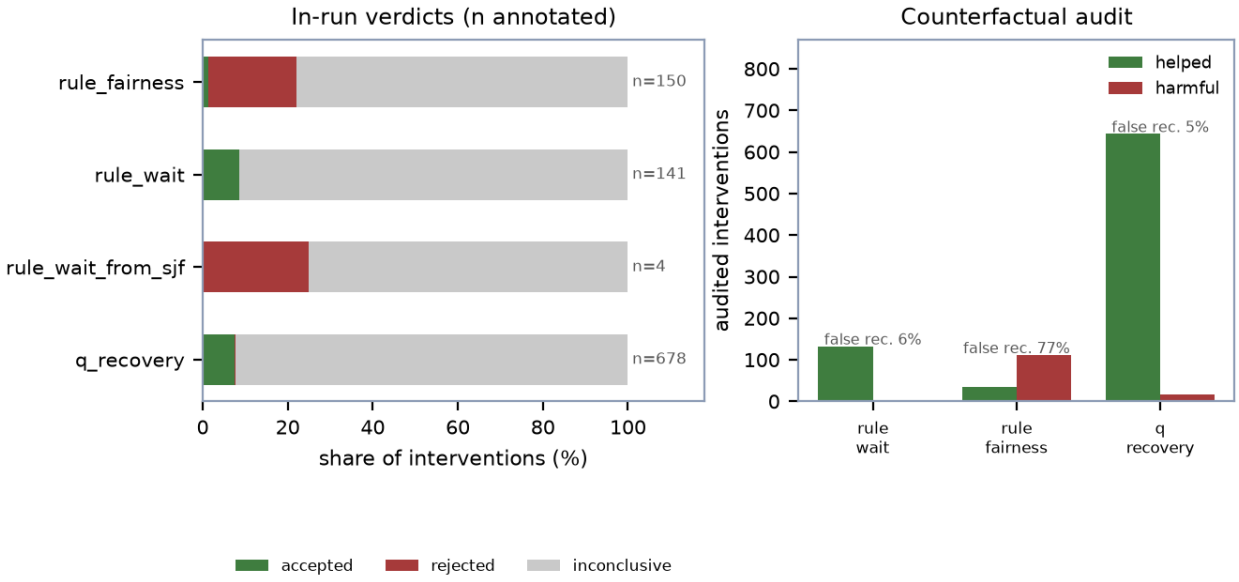


Figure 14 — Settlement outcomes per arm, and the counterfactually measured help rate of the same interventions.

The counterfactual audit answers the separate question of whether the actions were worth taking, which the in-run verdict cannot establish. For each intervention the same controller is replayed on the same trace with that one incident forced to hold:

Arm	Audited interventions	Mean marginal gain	Median	Helped	Harmful	False-recovery rate
rule_wait	141	+188.79	+172.0	133	3	5.7%
rule_fairness	150	-130.84	-110.5	35	112	76.7%
q_recovery	678	+192.07	+173.0	644	17	5.0%
rule_wait_from_sjf	4	-169.25	-158.5	0	4	100.0%

Table 14 — Matched-replay audit: the marginal value of each intervention in waiting-time units. A false recovery is an intervention whose matched gain is not positive.

Read together, the two tables describe the honest state of the system. The actions are genuinely useful — the waiting-time rule's interventions reduce total waiting time by about 188.79 units on average and 133 of 141 of them helped — while the in-run verifier certifies only a small minority of them and reports the rest as inconclusive. The verifier is therefore documented as a **safety filter with a measured rate**, not as a benefit estimator. The fairness rule shows the other side of the ledger: most of its interventions were harmful and three quarters of them were false recoveries, which is why the paired comparison of §7.4 places it behind the no-action control.

7.6 Verification quality on the held-out interventions

65 / 65	**8.0%**	**76.5%**	**0.951**
accepted interventions that were genuinely helpful	of genuinely helpful interventions cleared the accept boundary	of rejected interventions were harmful	rank AUC of the residual for “this intervention helped”

- **Accepted changes are trustworthy on this split.** All 65 accepted interventions were genuinely helpful (mean marginal gain 184.89 waiting-time units), against a base rate of 83.5% helpful among all audited interventions.
- **Rejection carries information.** 76.5% of the 34 rejected interventions were harmful, against a base rate of 14.0%.
- **But certification is rare.** Only 8.0% of the 812 genuinely helpful interventions cleared the accept boundary (54 clear even the unconditional null p95), so about 92% of them end inconclusive and are rolled back after the window. This is the central limitation of the current system and §9 states what would be needed to change it.
- **The statistic discriminates well even where it cannot certify.** The phase-adjusted residual ranks helpful above harmful interventions with AUC 0.951, against 0.613 for the raw closure statistic: the phase baseline is doing real work.

7.7 Detector quality

Scenario	Traces	Detected	Recall	Incidents / trace	Median delay (decisions)
arrival_storm	140	79	56.4%	0.57	6.0
burst_shock	140	105	75.0%	0.77	6.0
stable	140	96	68.6%	0.70	6.0

Table 15 — Controlled degradation scenarios on the validation split ('stable' traces are never injected).

Recall on injected degradations is 75% for a burst shock and 56% for an arrival storm, at a median delay of six decision intervals, while 0.70 events per stable trace is the error rate on the other side. Raising the congestion gate removes most false triggers and with them most true detections, so the trade-off is documented rather than optimised away.

7.8 Learning coverage

Training seed	Episodes	Incidents	Visited states	Visited pairs	% of declared table	% of reachable pairs
7101	1200	772	32	58	1.12%	6.7%
7102	1200	753	29	57	1.10%	6.6%
7103	1200	769	30	62	1.20%	7.2%
7104	1200	771	33	66	1.27%	3.8%
7105	1200	745	31	56	1.08%	6.5%

Table 16 — Coverage of the learned controller's table, per independent training seed.

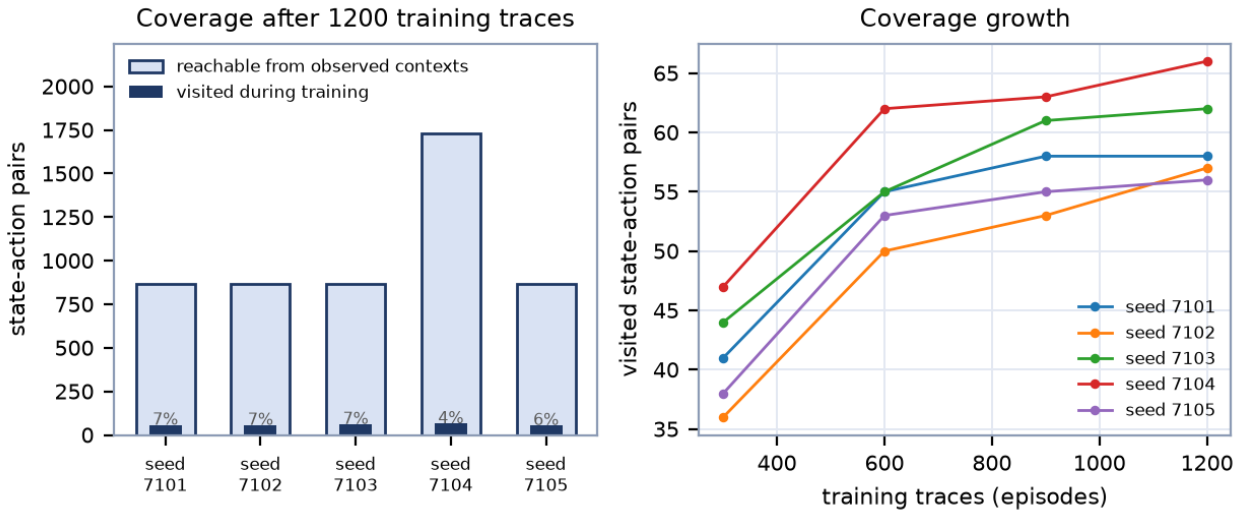


Figure 15 — Visited state-action pairs against the pairs reachable from the observed contexts, and their growth with training.

8. Implementation progress

The system described above is implemented and produces the evidence quoted in this report. This section states what is complete, shows the command that produces it, and reproduces real output rather than describing intended output.

8.1 Completion by lifecycle stage

Stage	Implementation	Status	Evidence
1. Reproducible workload	workload/generator.py	Complete	split-disjointness and reproducibility tests
2. Discrete-event simulation	scheduler/runtime.py	Complete	engine pinned to the four fixed schedulers
3. Causal telemetry	healing/monitor.py	Complete	monitor tests; reference-absorption rule
4. Degradation detection	healing/detector.py	Complete	detector tests; the injection scenarios of §7.7
5. Diagnosis	healing/diagnosis.py	Complete	diagnosis tests; diagnosis mix on the control
6. Recovery decision (rule / Q, masked)	healing/policies.py, rl/q_learning.py	Complete	policy tests; the coverage analysis of §7.8
7. Apply the action	healing/actions.py, healing/controller.py	Complete	no-op semantics tests

Stage	Implementation	Status	Evidence
8. Verify over a declared window	healing/verifier.py	Complete	verification tests; calibrated error rates
9. Accept / roll back / safe fallback	healing/controller.py	Complete	verdict and fallback tests; one fallback episode on the held-out split
10. Record and audit	incident ledger, counterfactual replay	Complete	audit with divergence checks (§7.5)

Table 17 — Lifecycle stages.

8.2 Automated verification of the implementation

The suite runs in about fifty seconds and covers the engine, the healing loop and the experiment harness, including end-to-end reproduction of the study on a small design:

```
$ python -m pytest -q
472 passed in 49.79s
```

Listing 3 — Test suite.

8.3 Demonstration 1 — the settlement stage on real runs

The following is the recorded behaviour of the settlement stage on the held-out split: 973 interventions across 4 healing arms, of which 65 were verified as improvements, 34 were confirmed failures and 874 ended inconclusive and were rolled back. Safe fallback fired in 1 episode(s), in the arm whose interventions are mostly harmful — which is the arm that ought to lose its adaptation budget.

8.4 Demonstration 2 — one trace, decision by decision

healing_demo.json records a complete worked trace, selected by a rule declared before the run and re-executed during artifact generation (the writes are refused if the replay does not reproduce the recorded incident count). The trace is:

- **Workload**: family *long_jobs*, validation repetition 0, fingerprint *2cc74cad24d7c29d*, arm *q_recovery* with training seed 7105.
- **Detection**: incident at decision 6 (t = 142), diagnosis *fairness_starvation*, window mean 48.00 against reference 27.75 and band 4.16.
- **Action**: *set_sjf* — the masked action set offered only actions that could change the live configuration.
- **Verification**: window closed after 5 decisions and 5 completions, with closure 2.324 against an expected closure of 2.010 for that trigger phase.
- **Verdict**: *accepted* (verified_phase_residual), residual +0.314 against the declared accept boundary 0.30 — the configuration was kept.
- **Cost and outcome**: switch overhead 5.00 units, reward -0.053 (the residual minus the boundary minus the declared intervention cost), 14 context switches and mean waiting time 137.67 for the trace.

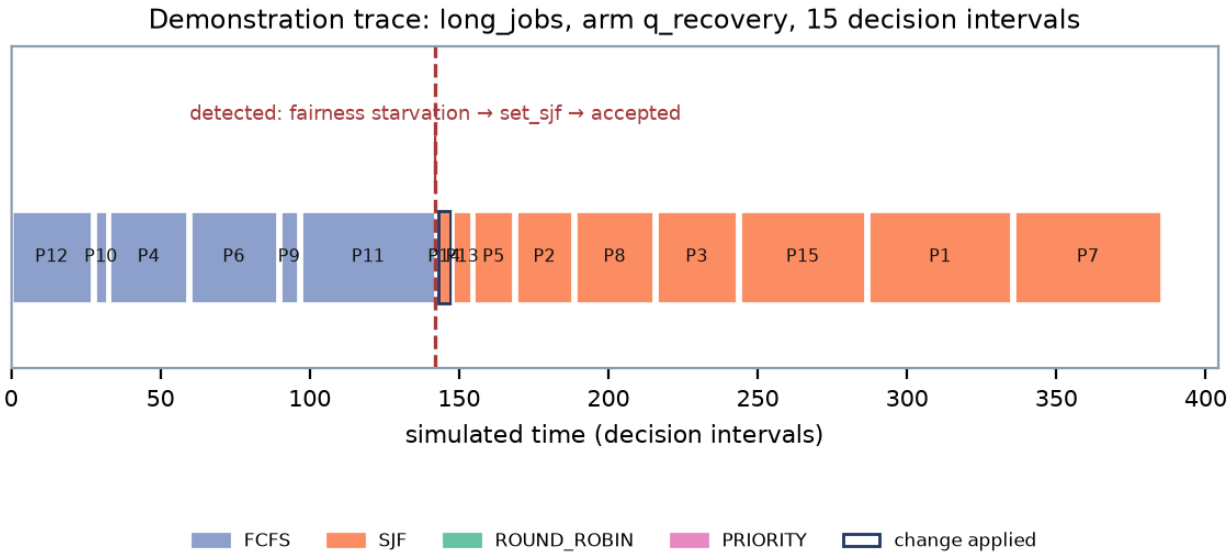


Figure 16 — The demonstration trace: the interval structure around the detection, the applied change and the closed window.

8.5 Demonstration 3 — artifacts produced by one command

A single command produces the 21 machine-readable artifacts and three figures that back this report:

```
$ python main.py healing-study
Final-test means
----------------
  arm                        wait      p95     resp   incid   intv
  fixed_fcfs               130.84   258.42   130.84    0.00   0.00
  fixed_sjf                 94.66   249.20    94.66    0.00   0.00
  fixed_round_robin        217.05   310.41    26.50    0.00   0.00
  fixed_priority           128.32   262.75   128.32    0.00   0.00
  detector_only            130.84   258.42   130.84    0.65   0.00
  rule_fairness            136.78   263.72   110.40    0.71   0.71
  rule_wait                122.39   251.88   122.39    0.67   0.67
  rule_wait_from_sjf        94.87   249.54    94.87    0.82   0.02
  q_recovery               122.56   251.98   122.54    0.67   0.65
  q_recovery_from_sjf       94.66   249.20    94.66    0.82   0.00

(abridged: the command also prints the split summary, the verification verdicts, the
matched counterfactual audit, the coordinate-descent detection sweep and the coverage
analysis, and writes the 21 artifacts and three figures.)
```

Listing 4 — Console output of the primary command, abridged to the held-out table (reproduced from the run behind this report).

Artifact	Content
healing_metrics_final_test.csv	per-trace metrics of every arm on 3780 rows
healing_incidents_final_test.csv	the incident ledger: 70 columns of evidence, action, window statistics, verdict, cost and reward
healing_audit.csv	one matched-replay row per intervention plus one never-heal row per episode, with the divergence check
healing_comparisons.csv	paired bootstrap differences and intervals (§7.4)
healing_threshold_derivation.csv	derived thresholds, gate sweep and the out-of-sample control check
healing_verification_models.csv	the measured comparison of candidate phase baselines, with paired uncertainty
healing_action_probe.csv	every action replayed at every probed incident: recorded reward against measured effect
healing_coverage*.csv	learning coverage and its growth with training length
healing_detection_*.csv	injection scenarios and per-arm detection summaries
healing_demo.json	the worked trace of §8.4, replay-verified
healing_summary.json	machine-readable summary of everything above; the only source of the numbers in this report

Table 18 — The artifacts behind this report.

An independent re-run into a separate directory reproduces all 21 artifacts and all three figures byte for byte, and the report itself is generated from those files, so a change in the system changes the report.

9. Challenges encountered and plan for completing the remaining work

9.1 Challenges encountered

- **Verifying a change is harder than making it.** The recovery actions do help — the audit measures roughly +189 waiting-time units per intervention for the rule and +192 for the learned controller — but only 8.0% of those helpful interventions clear the accept boundary, and about 92% end inconclusive. The cause is a signal-to-noise problem, not a coding defect: one window shows about one standard deviation of residual noise for an action whose effect moves the statistic by roughly the same amount, so a single window cannot certify an ordinary improvement at a fixed false-accept budget.
- **An earlier verification rule was genuinely wrong.** The first design compared the window's absolute improvement against the trace's normal level and certified 94.6 % of windows in which nothing had happened; the second collapsed every non-accepted window into a failure, which rolled back helpful changes and taught the learner that they were harmful. Both are documented as defects in *docs/DESIGN_AND_CHOICES.md* together with the fix, because the fix is where the tri-state verdict and the phase baseline came from.
- **Detector false triggers.** With no intervention at all the control declares 0.62 events per trace on the training control and 0.65 on the held-out control, and 0.70 per stable trace in the injection probe. A trace that begins with an arrival backlog is at its worst at the beginning, and the monitor cannot distinguish “this trace is hard” from “this configuration is bad” without a longer history than one trace provides.
- **The learned controller does not beat the hand-written rule.** The paired difference is +0.18 waiting-time units (-0.04 to +0.48), i.e. indistinguishable, and neither beats fixed SJF. Coverage explains part of it: 1200 training traces per seed visit only 29–33 of the 648 states, which is 3.8–7.2 % of the pairs reachable from the observed contexts, and the coverage curve is flat after about 900 episodes because the event supply, not the training budget, is the binding constraint.
- **Neither recovery method reaches SJF.** SJF is handed the one thing a real scheduler does not have, exact burst times, and on these workloads it is the strongest single policy (the held-out table of §7.4). The loop's value proposition is therefore adaptation to a configuration it was not given, not superiority to an oracle.
- **The safety fallback fired once.** At most six interventions per trace and two consecutive *confirmed* failures disable adaptation; on the held-out split that combination occurred once, in the fairness arm, and once on validation. The mechanism is unit-tested, but its effect over a population of traces is a single observation rather than a measured distribution.
- **Simulator assumptions.** One CPU, one FIFO queue, constant switching cost, no I/O, no interrupts, stationary arrivals within a trace, and visible burst times. Nothing in the repository is a claim about a real kernel.

9.2 What would move the needle, in order of expected value

- **1. More evidence per action (W1).** Instead of one 5-decision window per intervention, accumulate several windows or a sequential test on the same statistic, and stop early when the residual crosses a boundary. The audit already measures the headroom: the paired spread of the statistic against its own no-action replay is smaller than the spread of the phase model's residual, so repeated evidence should convert part of the inconclusive mass into verdicts without loosening the false-accept budget.
- **2. A better phase reference (W2).** The declared baseline is a static least-squares model of the no-action post-window load. The specification search in *healing_verification_models.csv* already compares eight candidates against it with paired uncertainty; the two directions that remain open are a per-trace drain model fitted from the trace's own completed-work history, and conditioning on the monitor window's maturity rather than only on the trigger phase.
- **3. A more informative reward (W3).** The reward currently uses the same statistic as the verdict, which keeps learning honest but inherits its noise. The action probe shows that the canonical `controller_reward` already ranks actions correctly *within* an incident (mean Spearman rank correlation 0.449, median 0.531, and its argmax is the best action in 64 of 98 canonical probed incidents — 65.3 %; legacy raw proxy on 21 incidents gave 0.622 mean, 0.875 median, 15 of 21); the remaining gain is in reducing between-incident variance rather than in re-shaping the sign.
- **4. More useful incidents, or a smaller state space (W4).** Coverage is event-limited. Options with evidence behind them are to train on a larger number of shorter traces, to reduce the state space by removing the two features that explain least variance, or to fall back to the rule in states without an action comparison — the last is already counted and reported (11% of decisions on average).

- **5. Detector calibration (W5).** The congestion-gate sweep is recorded in the threshold artifact; a two-sided gate (fire only when the queue is congested *and* the trace is still arriving) is the natural next experiment, but it must be chosen on training and validation data, as the current gate was.
- **6. An enriched action space (W6).** The declared action space contains five actions that cannot change anything from the standard starting configuration, so the real choice is among three. Adding actions with a modelled effect (for example age-aware boosting) would have to come with the mechanism to evaluate them, otherwise they are names rather than actions.

9.3 Sequence for the remaining work

Step	Work item	Gate before it is reported
Next	W1 repeated-window verification (implementation + unit tests)	false-accept rate on the training control stays within the declared budget
Next	W2 per-trace drain model, compared against the declared baseline	paired comparison in the verification-model artifact, validated out of sample
Then	W4 coverage experiment (state-space reduction or longer training)	coverage and paired result measured on validation only
Then	W5 two-sided congestion gate	false triggers and recall both reported on the injection scenarios
Then	W3 reward variance reduction	within-incident rank correlation and audit agreement must not degrade
Then	A frozen re-run on a newly declared held-out seed	the design is frozen first; the new seed is declared before the run

Table 19 — Remaining work and the gate each step must pass.

Two rules govern this plan. Any change is developed on training and validation data only, and any change that survives is followed by a frozen run on a freshly declared held-out seed, because the current held-out stream will have been examined during development and can no longer serve as untouched evidence. If validation shows that the available observation window cannot separate useful actions from noise, the honest outcome is to keep the conservative rule and report the limit, which is what the current release does.

Appendix A — Commands, seeds and preserved workflows

A.1 Exact reproduction commands

```
# environment (Python 3.11)
python3 -m venv .venv && .venv/bin/python -m pip install -r requirements.lock

# primary system: training, held-out evaluation, counterfactual audit, figures (~47 s)
.venv/bin/python main.py healing-study

# reproduce without touching the committed evidence
.venv/bin/python main.py healing-study --results-dir /tmp/healing-results \
                                       --figures-dir /tmp/healing-figures

# validation-only threshold derivation (refuses the final-test split by construction)
.venv/bin/python -m experiments.healing_thresholds

# the test suite
.venv/bin/python -m pytest -q                       # 472 passed

# this report, from the committed artifacts
.venv/bin/python -m pip install -r requirements-report.txt
.venv/bin/python tools/build_review2_report.py      # PDF
.venv/bin/python tools/build_review2_docx.py        # editable DOCX
```

Listing 5 — Reproduction commands.

A.2 Seeds in force

Seed	Value	Role
Training	7101, 7102, 7103, 7104, 7105	one independently trained controller each, 1200 traces per seed
Validation	8201	development decisions, demonstration trace, out-of-sample threshold checks
Final test	23819	declared before the frozen run; evaluated once
Bootstrap	104729	1000 replicates for every paired interval
Retired	20501, 20719, 21617, 72601, 21977 (and 9401)	examined during development; refused as a held-out seed by the configuration

Table 20 — Seed hygiene.

A.3 Preserved secondary workflows

Two earlier systems are kept in the repository because they document the path to the current design and are still runnable. They are **not** the project system and their numbers must not be mixed with the healing study's:

- **Offline policy-selection study** (*main.py study*): chooses one policy per *complete* workload from workload features before execution, with no reaction, no verification and no rollback. Its own seed stream produces the following held-out objective costs (lower is better):

Arm	Mean objective cost	Mean regret
best_fixed	0.8419	0.1040
fingerprint_rule	0.7752	0.0372
ql_orig	0.7963	0.0524
ql_orig_fingerprint	0.7838	0.0405
ql_risk	0.7953	0.0574
ql_risk_origstate	0.8253	0.0874
random	0.9778	0.2398

Table 21 — Offline selector, held-out mean objective cost and regret.

- **Non-healing runtime appendix** (*main.py runtime-experiment*): the event engine with a per-epoch Q-learning controller and no healing loop. Mean waiting time on its own held-out split reads:

Method	Mean waiting time
SJF	93.345
Runtime Q-learning	104.789
FCFS	127.797
Priority	129.207
Causal heuristic	203.404
Round Robin	215.729

Table 22 — Runtime appendix, mean waiting time.

Both workflows reproduce their committed artifacts (the runtime appendix differs only in wall-clock timing columns, which are machine-dependent and are never used as results). Keeping them costs nothing in terms of the primary system, and the runtime engine is the same simulator the healing loop drives.